Homework 3

Moida Praneeth Jain
2022101093
October 6, 2025

1) Question 1

1 (a) Maximum File Size in Unix File System

Unix and Linux file systems use an inode structure to manage file metadata and locate a file's data blocks on the disk. An inode stores a file's permissions, owner, timestamps, and pointers to the data blocks (file contents).

Assumptions:

- Block size = $4\ \mathrm{KB}$
- Pointer size = $4\ \mathrm{Bytes}$
- Inode structure as follows:
  - 12 direct pointers
  - 1 single indirect pointer
  - 1 double indirect pointer
  - 1 triple indirect pointer

Now, to find the maximum file size, we calculate the contributions from each part of the inode structure.

- Direct pointers:
  - Each direct pointer points to a block of size $4\ \mathrm{KB}$.
  - Total from direct pointers = $12 * 4\ \mathrm{KB} = 48\ \mathrm{KB}$.
- Single indirect pointer:
  - A single indirect pointer points to a block that contains pointers to data blocks.
  - Number of pointers in a block = $\frac{4\ \mathrm{KB}}{4\ \mathrm{Bytes}} = 1024$.
  - Total from single indirect pointer = $1024 * 4\ \mathrm{KB} = 4096\ \mathrm{KB} = 4\ \mathrm{MB}$.
- Double indirect pointer:
  - A double indirect pointer points to a block that contains pointers to single indirect blocks.
  - Each single indirect block can point to 1024 data blocks.
  - Total from double indirect pointer = $1024 * 1024 * 4\ \mathrm{KB} = 4\ \mathrm{GB}$.
- Triple indirect pointer:
  - A triple indirect pointer points to a block that contains pointers to double indirect blocks.
  - Each double indirect block can point to 1024 single indirect blocks, and each single indirect block can point to 1024 data blocks.
  - Total from triple indirect pointer = $1024 * 1024 * 1024 * 4\ \mathrm{KB} = 4\ \mathrm{TB}$.

Adding all these contributions together gives the maximum file size:

Total maximum file size = $48\ \text{KB} + 4\ \text{MB} + 4\ \text{GB} + 4\ \text{TB}$, which is approximately $4\ \text{TB}$.

1 (b) Why GFS Does Not Use a Hierarchical Inode Mechanism

GFS does not use such a mechanism because it is designed for a completely different use case. The linux file system is designed for general users, not industry scale workloads, whereas GFS was designed for large scale distributed systems. The hierarchical inode mechanism is not suitable for GFS because:

- **Huge file sizes and limited metadata**: GFS manages huge files by dividing them into large chunks (typically 64 MB). Because of this, the amount of metadata needed to be stored is much lower than the linux file system that has 4 KB blocks.
- **Central Master Node**: GFS uses a single master node to store and manage all the metadata. A hierarchical inode structure will be inefficient for this master to traverse and do operations on. This master simply has a lookup table that maps file paths directly to the chunk handles.
- **Performance and Scalability**: GFS is primarily optimized for sequential appends and large streaming reads. The inode structure is not optimized for this at all, as traversing the levels of indirect pointers to find a particular data block will lead to huge bottlenecks in performance. In GFS, the master can quickly provide chunk locations to the client, and the client can communicate directly with the chunk servers.

1 (c) Flat vs. Hierarchical Metadata

Advantages of Hierarchical storage model:

- The directory tree structure is intuitive to users and easy to organize data as files and folders.
- This allows for fine grained access control by setting permissions at different levels of the hierarchy.

Disadvantages of Hierarchical storage model:

- Traversing the hierarchy can be slow, especially for deep directory structures.
- Managing and maintaining the hierarchy can be complex, especially with many files and directories.

Advantages of Flat storage model:

- Flat storage is simpler to implement and manage, as there are no nested structures to maintain.
- Flat storage is more scalable for large datasets, as it avoids the overhead of traversing a hierarchy.
- Flat storage results in better performance for operations like search, as there is no need to navigate through a directory tree.

Disadvantages of Flat storage model:

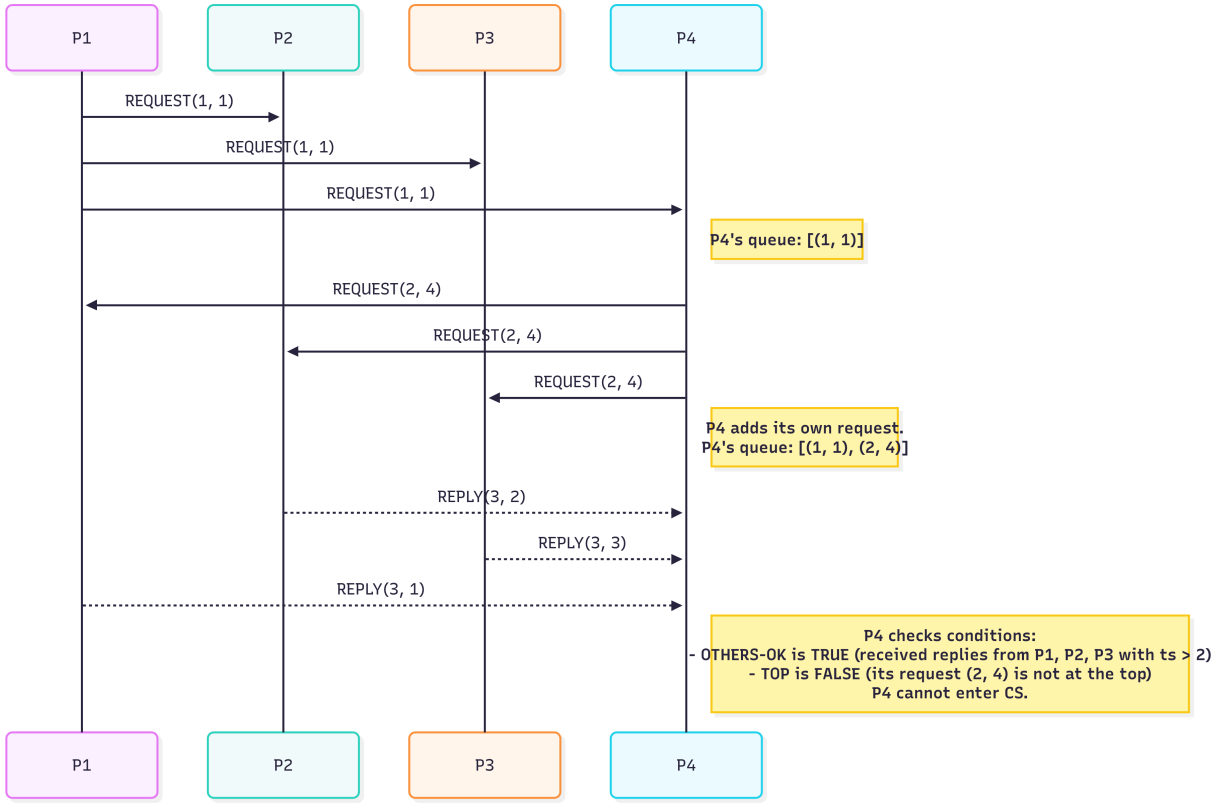
- Flat storage leads to a disorganized file system, making it difficult to find and manage files. It is not very user friendly.

- Flat storage usually does not provide the same level of access control as hierarchical storage, as permissions are set at the file level rather than at different levels of a hierarchy (since there is no hierarchy).

2) Question 2

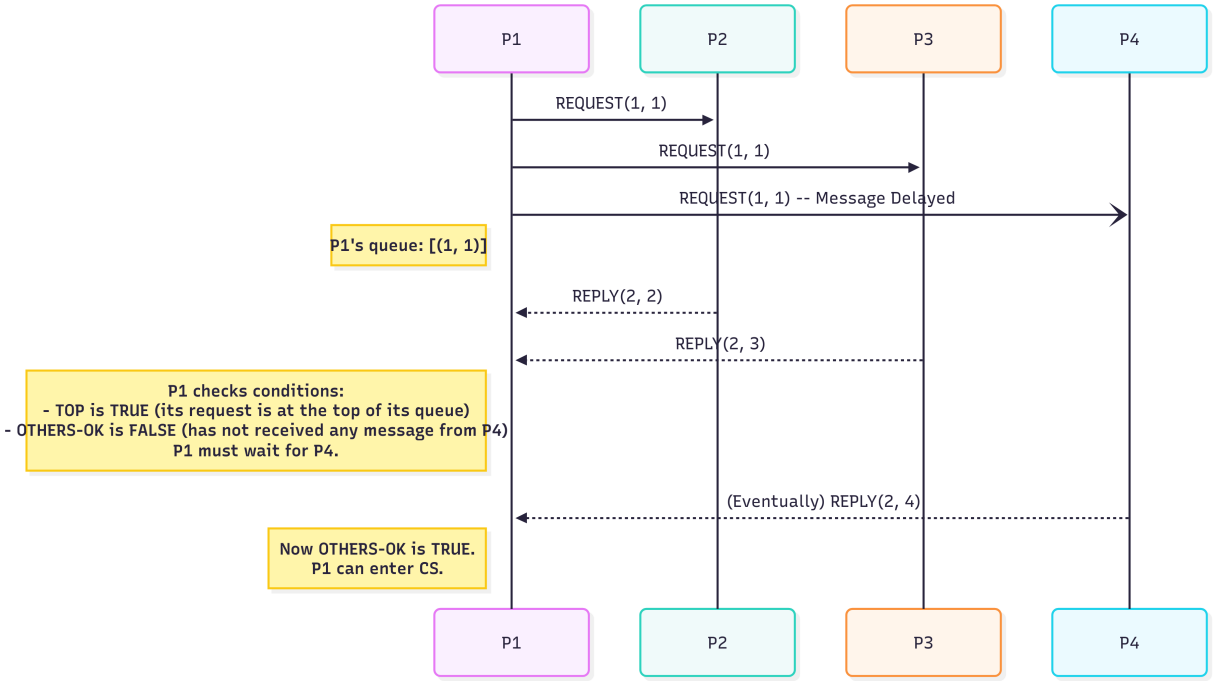
2 (a) OTHERS-OK but not TOP

The key here is that P4 makes its request after receiving an older, higher-priority request from P1. Although P4 receives timely replies from all other processors for its own request, its local request queue has P1′s older request at the top. This correctly prevents P4 from entering the critical section. After P1 receives all the replies and it is done with the critical section, it sends a release message to P4. Upon receiving this release message, P4 can then enter the critical section since the TOP condition would then be satisfied.



2 (b) TOP but not OTHERS-OK

P1 broadcasts its request, and its request is at the top of its queue, making TOP true. But, due to a significant delay in the message channel to P4, P1 has not yet received any message back from P4. Without hearing from P4, P1 cannot be sure that P4 doesn’t have an older, unknown request, so the OTHERS-OK condition fails, and P1 must wait.



3) Question 3

For a system with n nodes with a maximum of f possible crash failures, the algorithm runs for exactly $f + 1$ rounds, where in each round:

- Each process broadcasts its current value to all other processes.
- Each process collects all values it receives in that round.
- Each process updates its own value to be the minimum of its current value and all the values it just received.

After $f + 1$ rounds, all non-faulty processes will have the same minimum value (the consensus value).

Let us consider an example with $n = 6$ nodes: $P_1, P_2, P_3, P_4, P_5, P_6$ with initial values

$$\{P_1 : 10, P_2 : 20, P_3 : 10, P_4 : 20, P_5 : 5, P_6 : 3\}$$

where say P_5, P_6 are the faulty nodes ($f = 2$).

Round 1

Let's say P_5 sends its value 5 only to P_1 and then crashes, and P_6 crashes before sending its value 3 to any node.

- $P_1 : \min(10, \{20, 10, 20, 5\}) = 5$
- $P_2 : \min(20, \{10, 10, 20\}) = 10$
- $P_3 : \min(10, \{10, 20, 20\}) = 10$
- $P_4 : \min(20, \{10, 20, 10\}) = 10$

Round 2

- $P_1 = \min(5, \{10, 10, 10\}) = 5$
- $P_2 = \min(10, \{5, 10, 10\}) = 5$
- $P_3 = \min(10, \{5, 10, 10\}) = 5$
- $P_4 = \min(10, \{5, 10, 10\}) = 5$

Round 3 All nodes have the same value now, so they will all remain at 5.

In the worst case, the algorithm requires $f + 1$ rounds to ensure that all non-faulty nodes reach consensus. But in this example, consensus was reached in just 2 rounds. The number of rounds to *achieve* the consensus value depends on the initial values and the pattern of message delivery, but the algorithm guarantees that consensus will be reached within $f + 1$ rounds regardless of these factors.

If the faulty nodes crash without sending any messages in round 1, then consensus will be achieved in just one round. In this example, this consensus value will be 10, as the values 5 and 3 were never introduced to any non-faulty node.

If in round 1, P_6 sends 3 to only P_5 and then crashes, and then in round 2, P_5 sends 3 to only P_1 and then crashes, then consensus will be achieved in 3 rounds with the value 3. This is the worst case scenario for this example, as the faulty nodes are able to propagate their minimum value to one node in each round before crashing.

4) Question 4

For a system with n nodes and a maximum of f Byzantine failures, the phase king algorithm works in $f + 1$ phases, where in each phase:

- Round 1: All processors exchange their current preferences. Each processor determines a local majority value and counts its occurrences.
- Round 2: A designated “king” (may change in different phases) broadcasts its majority value. Processes with the number of occurrences greater than $\frac{n}{2} + f$ keep their own value, whereas others adopt the king’s value.

The correctness is determined by the nonfaulty king lemma and the unanimous phase lemma. The unanimous phase lemma states that if all non-faulty processes start a phase with the same preference, the will also end the phase with that same preference. This implies $n - f > \frac{n}{2} + f$, which simplifies to $n > 4f$.

Thus, this algorithm will fail if $n \leq 4f$. As per the question, let us consider a case where $n < 4f$. We will take $n = 7$ and $f = 2$

Let’s say the nodes P1,P2,P3,P4,P5 are correct, while P6,P7 are byzantine. The threshold for a majority is $\frac{n}{2} + f = 3.5 + 2 = 5.5$, so a process needs at least 6 votes to keep its own value.

Consider the case where after a previous phase with a non-faulty king, all correct nodes have reached a consensus and are starting a new phase k. The king of this phase is P6 (faulty).

Start of phase k

All correct nodes (P1, P2, P3, P4, P5) have the same preference value $\text{pref} = V$

Phase k Round 1

All correct nodes broadcast their preference V.

Both the faulty nodes send a value $!V$ to P1, and value V to P2, P3, P4, P5.

P1 receives V from P2,P3,P4,P5, and $!V$ from P6,P7. P1 has majority of V with multiplicity 5 (including its own)

P2,P3,P4,P5 receive V from the other 6 nodes each. They get V with a multiplicty of 7 (including their own)

Phase k Round 2

The king P6 broadcasts its value $!V$ to all nodes.

P1 has a weak majority ($5 < 6$) so it adopts the king's preference $!V$.

P2,P3,P4,P5 have a strong majority ($7 > 6$) so they keep their preference V.

End of phase k At the end of phase k, P1 has preference $!V$, while P2,P3,P4,P5 have preference V. The correct nodes have not reached consensus.

Clearly, the agreement property of consensus is violated here. The correct nodes started in perfect agreement, but have now been split up by the byzantine nodes. This shows that the phase king algorithm fails when $n < 4f$.